UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE CIENCIAS, MATEMATICAS Y FISICAS

CARRERA DE SOFTWARE

MATERIA:

APLICACIONES DISTRIBUIDAS

TEMA DE ACTIVIDAD:

ARQUITECTURA P2P

DOCENTE:

CALDERON GAVILANES MARLON ADRIAN

ESTUDIANTE:

TENESACA PINCAY CINTHIA MARIA

CICLO ESCOLAR:

2025 – 2026 CICLO 2

INDICE

1. BitTorrent Protocol 1

1.1. Lenguaje de programación utilizado 1

1.2. Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida) 1

1.3. Casos de uso 1

1.4. Ventajas y Desventajas 2

1.5. Nivel de complejidad 2

2. Gnutella 2

2.1. Lenguaje de programación utilizado 2

2.2. Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida) 2

2.3. Casos de uso 3

2.4. Ventajas y Desventajas 3

2.5. Nivel de complejidad 3

3. Ethereum / Blockchain (enfoque P2P) 3

3.1. Lenguaje de programación utilizado 3

3.2. Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida) 4

3.3. Casos de uso 4

3.4. Ventajas y Desventajas 4

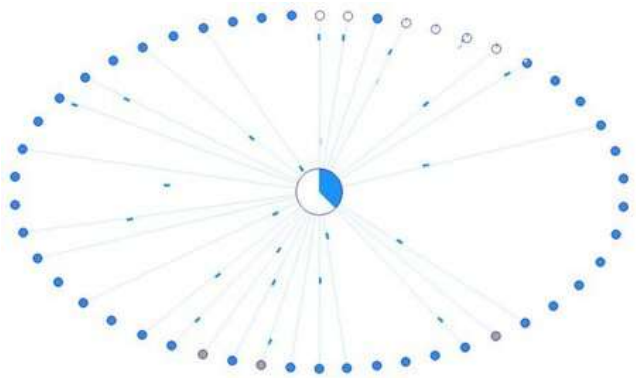
3.5. Nivel de complejidad 4

Bibliografía 5

HERRAMIENTOSD O TECNOLOGIAS P2P

1. BitTorrent Protocol

El BitTorrent es un protocolo de intercambio de datos de forma descentralizada. Es un sistema que siempre ha gozado de mucha popularidad para intercambiar archivos de todo tipo, y aunque la descarga directa le quitó protagonismo durante unos años, ahora vuelve a ser un protocolo muy utilizado. Hay páginas como las de distribuciones GNU/Linux y algunas instituciones que lo utilizan para permitirte descargar sus archivos.



1.1.Lenguaje de programación utilizado

El protocolo BitTorrent fue desarrollado originalmente en Python por Bram Cohen en 2001. Aunque la especificación técnica es independiente del lenguaje, Python se utilizó para la primera implementación (Mainline). Hoy en día, los clientes se escriben en diversos lenguajes como C++, Go, Rust, Java o Python.

1.2.Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida)

BitTorrent emplea una arquitectura híbrida: usa trackers centrales (servidores) para coordinar el descubrimiento inicial de peers, pero las transferencias de datos ocurren directamente entre pares descentralizados. Esto combina estructura (trackers, DHT en extensiones) con no estructurada (swarms mesh).

1.3.Casos de uso

- Distribución de software y OS como Linux ISOs, reduciendo carga en servidores oficiales.
- Datasets científicos (NASA imágenes, BioTorrents para biología) y recuperación de desastres gubernamentales.
- Contenido multimedia independiente (películas documentales) y almacenamiento descentralizado vía BTFS.

1.4.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
Eficiencia: bajo costo para proveedores, alta redundancia y resistencia a flash crowds.	***Exposición IP:*** riesgos de privacidad sin VPN, monitoreo de actividad.
Escalabilidad: peers comparten carga, velocidades altas con más usuarios.	***Malware:*** archivos infectados comunes en trackers públicos.
Resiliente: descentralizado para datos grandes/censurados.	***Dependencia inicial en trackers;*** seeding bajo reduce velocidad.

1.5.Nivel de complejidad

Media para implementaciones básicas: el protocolo es simple (TCP-based, specs claras), con clientes en múltiples lenguajes vía libs existentes. Alta para features avanzadas como DHT o congestión personalizada (requiere UDP/TCP handling). Ideal para devs con experiencia en redes, no para principiantes absolutos.

2. Gnutella

2.1.Lenguaje de programación utilizado

Originalmente implementado en Java (cliente inicial de Nullsoft/AOL en 2000), con protocolo basado en mensajes binarios sobre TCP. Clientes modernos usan C++, Java, Python y otros; su especificación abierta permite ports en múltiples lenguajes.

2.2.Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida)

Es uno de los sistemas peer-to-peer más conocidos y populares dentro de la categoría no estructurada. Gnutella fue diseñado para ser un sistema de intercambio de archivos basado en una superposición P2P no estructurada que permite el intercambio de archivos descentralizado, escalable, confiable y anónimo entre los nodos participantes. Gnutella basa su popularidad en su simplicidad y su membresía abierta y flexible, lo cual, sin embargo, ha provocado graves problemas de degradación del rendimiento .

2.3.Casos de uso

- Intercambio descentralizado de archivos multimedia y software libre.
- Búsquedas distribuidas anónimas y resilientes (e.g., BearShare, LimeWire).
- Prototipo para redes P2P escalables sin autoridad central.

2.4.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
Descentralización total: resistente a fallos y censura, membresía abierta.	***Degradación rendimiento:*** flooding genera alto tráfico (TTL~7 limita a ~10k nodos).
Simplicidad y anonimato: inteligencia distribuida en búsquedas.	***Escalabilidad pobre en redes grandes;*** leaf nodes sobrecargan ultrapeers.
Flexibilidad: enjambre dinámico, TTL/horizonte evita loops.	***Seguridad baja:*** exposición IP, spam queries y ataques de inundación.

2.5.Nivel de complejidad

Media-alta: protocolo simple (mensajes Ping/Pong/Query/QueryHit/Push), pero implementar flooding, roteo dinámico y manejo TTL requiere experiencia en redes TCP y concurrencia. Fácil para prototipos académicos en Java, compleja para optimizaciones anti-spam/escala.

3. Ethereum / Blockchain (enfoque P2P)

3.1.Lenguaje de programación utilizado

Go (Golang) es el principal para el cliente oficial Geth, con implementaciones en Rust (OpenEthereum/Erigon), Python, JavaScript (ethers.js/web3.js para interacción) y C++. El protocolo devp2p es lenguaje-agnóstico, pero Go domina el core networking.

3.2.Tipo de arquitectura P2P (Estructurada, no estructurada, hibrida)

Estructurada: Emplea Kademlia DHT (K-buckets) para descubrimiento de nodos, discovery v5 (discv5) para bootstrap y enrutamiento basado en distancias XOR, combinado con propagación gossip en subgrafos. Forma scale-free networks con small-world properties.

3.3.Casos de uso

- Ejecución de smart contracts y DApps descentralizadas (DeFi, NFTs).
- Transferencias de valor (ETH) y consenso proof-of-stake vía propagación de bloques/transacciones.
- Sincronización de estado global (ledger) en miles de nodos independientes.

3.4.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
Alta resiliencia: Tolera fallos/maliciosos vía consenso distribuido.	***Alto consumo recursos:*** Nodos full requieren TB de storage y ancho de banda.
Descubrimiento eficiente: Kademlia escala a millones de peers.	***Latencia propagación:*** ~12s para bloques críticos en picos.
Seguridad criptográfica: Firmas, encriptación RLPx en conexiones P2P.	***Complejidad consenso:*** Sincronización state expensive en redes grandes.

3.5.Nivel de complejidad

Alta: requiere manejar DHT, gossip protocols, validación EVM y consenso (PoS post-Merge); ideal para expertos en redes distribuidas. Usar clientes como Geth simplifica, pero custom nodes demandan conocimiento profundo de devp2p/RLP.

Bibliografía

Abril. (29 de Noviembre de 2018). *Protocolo Gnutella y Servicios p2p*. Obtenido de scribd: https://es.scribd.com/presentation/394450707/Protocolo-Gnutella-y-Servicios-p2p

BitTorrent (software). (s.f.). Obtenido de wikipedia: https://en.wikipedia.org/wiki/BitTorrent_(software)#:~:text=Historia,-editar&text=El%20programador%20Bram%20Cohen%20dise%C3%B1%C3%B3,no%20es%20de%20c%C3%B3digo%20abierto%20.

de Olveira Costa, F. L. (s.f.). *Mapping the Gnutella Network*. Obtenido de Slide: https://www.ic.unicamp.br/~bit/ensino/mo809_1s13/gnutella.pdf

Ethereum. (s.f.). Obtenido de sciencedirect: https://www.sciencedirect.com/topics/computer-science/ethereum

Ethereum – Arquitectura y componentes. (23 de Febrero de 2021). Obtenido de BLOCKKGENI: https://blockgeni.com/ethereum-architecture-and-components/

Fernández, Y. (8 de Febrero de 2021). *BitTorrent: qué es y cómo funcionan los torrents*. Obtenido de Xataka Basics: https://www.xataka.com/basics/bittorrent-que-como-funcionan-torrents

John F. Buford, H. Y. (2015). *Estudio de vanguardia sobre redes superpuestas P2P en entornos de computación ubicua*. Obtenido de sciencedirect: https://www.sciencedirect.com/topics/computer-science/gnutella-protocol

Los multiples usos del protocolo BitTorrent. (s.f.). Obtenido de BitTorrentVPN: https://www.bittorrentvpn.com/torrenting-uses/

Saluja, K. (23 de julio de 2025). *¿Es seguro BitTorrent? Guía completa para descargar torrents en 2025*. Obtenido de Spocket: https://www.spocket.co/blogs/is-bittorrent-safe

Todo lo que necesitas saber sobre BitTorrent. (s.f.). Obtenido de smartscribs: https://smartscribs.com/everything-you-need-to-know-about-bittorrent/